Practical No : 01

- Aim :

Write a program non-recursive and recursive program to calculate fibonacci numbers and analyze their time & space complexity.

- Prerequisite :

1. Basic of python programming
2. Concept of recursive and non-recursive functions
3. Execution flow of calculate fibonacci numbers.

- Theory :

Introduction to fibonacci numbers

- Introduction of fibonacci numbers means the fibonacci numbers series, named after Italian mathematician Leonardo. pisano Bongollo, later known as fibonacci no denoted as a fn. The numbers in fibonacci sequence are given as :
0, 1, 1, 2, 3, 5, 8, 13, 21, 38 ,.....

- In fibonacci series, every term is the sum of the precondition precending two terms, starting from 0 and 1 as

first & second terms in some old references. the term '0' might be omitted.

- What is the fibonacci series?

- The fibonacci series is the sequence of numbers, where every number is the sum of the preceding two numbers such that the first two terms are '0' & '1'.

- In some older versions of the series the term '0' might be omitted A fibonacci series can thus be given as 0, 1, 1, 2, 3, 5, 8, 13, 21, 34---
It can be thus be observed that every term can be calculated by adding the two terms before it.

- Given the first term, f0 and second term f1 as '0' and '1' the third term here can be given as $f2 = 0+1 = 1$
similarly

$f3 = 1+1 = 2$
$f4 = 2+1 = 3$

Given a number n point n-the fibonacci number.

- Fibonacci Sequence Formula :

- The fibonacci sequence of numbers 'fn' is defined using the recursive relation with the seed value $f_0 = 0$ & $f_1 = 1$

$f_n = f_{n-1} + f_2$

- Here the sequence is defined using two different part, such as kick-off and recursive relation the kick-off part is $f_0 - 0$ and $f_1 = 1$.

- The recursive relation part is $f_n = f_{n-1} + f_{n-2}$

It is noted that the sequence starts with 0 rather than 1. so f_5 should be 6th term of the sequence.

- Method 1 (use - non-recursion)

A simple method that is direct recursive implementation of mathematical recurrence relation is given above.

First we will store 0 & 1 in $f[0]$ & $f[1]$ respectively

- Next We'll iterate through array positions 2 to n-1 At each position i. we store the sum of the two preceding array values in

f[i].
Finally, we return the value of f[n-1] giving us the number at position n in the sequence. Here's visual representation sentation of this process.

- Time & space complexity of space optimized method.
- The time complexity of the fibonacci series is T(N) i.e. linear. We have to find the sum of two terms and it is repeated n times depending on the value of n.
- The space complexity and space complexity of dynamic programming g is O(1).

- Time & space complexity of dynamic programming.

- The time complexity of the above code is T(N), i.e. linear we have to find the sum of two terms & it is repeated n times depending on the value of n.
- The space complexity of the above code is O(N).

- Method 2 (case Recursion).

- lets start by defining f(n) as the function that returns the value of fn. To evaluate f(n) for (n>1) We can reduce our problem into two smaller problems of the same kind; f(n-1) & f(n-2) We can further

reduce f(n-1) and f(n-2) to f((n-1)-1) and f((n-1)-2) and f(n-2)-2), respectively.

If we repeat this reduction, we'll eventually reach our known base cases and, thereby obtain as solution to f(n).

Employing this logic, our algorithm for f(n) will have two steps.

1) check if n≤1, if so, return n.
2) check if n>1, if so, call our function f with inputs n-1 and n-2 and return the sum of two results.

Here's a visual representation of this algorithm

- Time and space complexity

- The time complexity of the above code is T(2^N), i.e. exponential

- The space complexity of the above code is O(N) for a recursive series.

Method	Time complexity	Space complexity
using recursion	T(n) = T(n-1) + T(n-2)	O(n)
Using DP	O(n)	O(1)

• Non-Recursive Fibonacci Program:-

```
def fibonacci - iterative (n):
    a, b = 0, 1
    if n == 0:
        return a
    elif n == 1:
        return b
    for - in range (2, n+1)
        a, b = b  a+b
    return b
n= int (input ("Enter the term (n);"))
print (f" fibonacci ({n}) = {fibonacci-iterative (n)}")
```

• output : Enter the term (n): 10
Fibonacci (10) = 55

• Recursive fibonacci program:

```
def fibonacci - recursive (n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
    return fibonacci - recursive (n-1) + fibonacci -
                    recursive (n-2)
n= int (input ("Enter the term (n);"))
point (f "fibonacci ({n}) = {fibonacci-recursive
    (n)}")
```

• output :

Enter the term (n): 12
Fibonacci (12) : 144

Space optimization of DP	O(n)	O(1)
using the power of matrix method	O(n)	O(1)
optimized matrix method	O(log n)	O(log n)
Recursive method in O time	O(log n)	O(n)
using direct formula	O(log n)	O(1)
DP using memoi -zation	O(n)	O(1)

- Application of fibonacci series:

- It is used in grouping of numbers and used to study different other special mathematical sequences.

- its find in coding (compute algorithms distributed system etc).

- Conclusion: In this way we have explored concept of fibonacci series using recursive & non-recursive method.

Practical No: 02

- Aim : Write a program to implement Huffman Encoding using a greedy strategy.

- Prerequisite :

1) Basic of python or Java programming.
2) concept of Greedy method.
3) Huffman Encoding concept.

- Theory :

1) Greedy Method :

- A greedy algorithm is an approach for solving a problem by selecting the best option available at the moment. It doesn't worry whether the current best result will bring the overall optimal result.

- The algorithm never reverse the earlier decision even if the choice is wrong it works in a top-down approach.

- Advantages of Greedy approach.

- The algorithm is easier to describe.

- This algorithm can perform better than other algorithms.

- Drawback of Greedy Approach:
- Greedy Algorithm does not always produce the optimal solution.
- Greedy Algorithm:

1) To begin with, the solution set is empty.

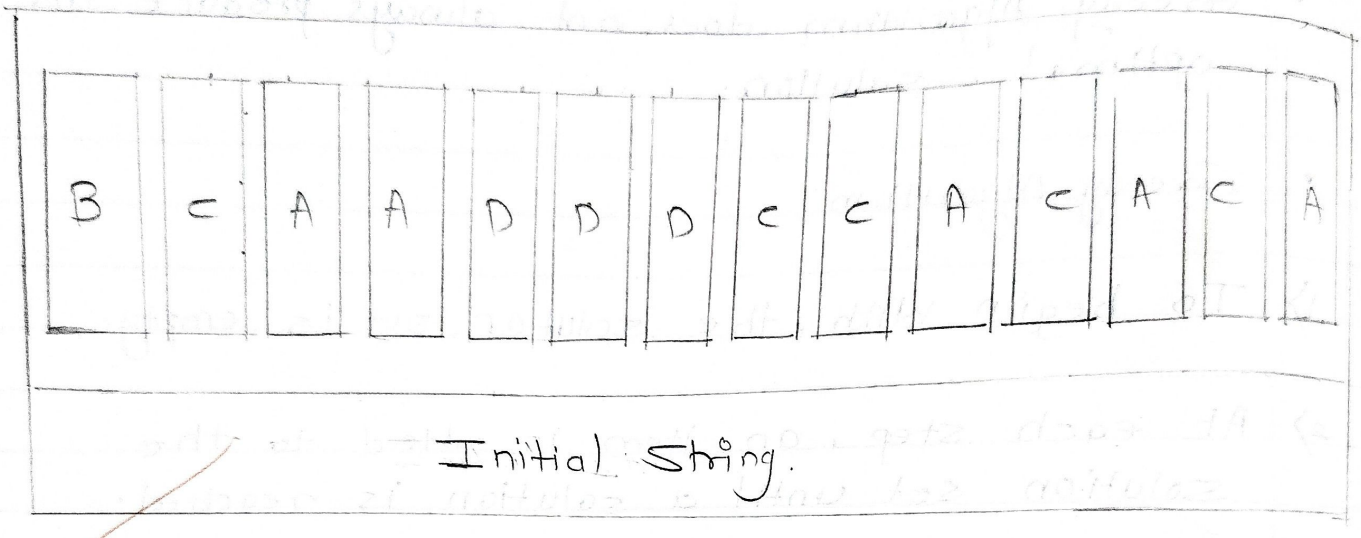
2) At each step, an item is added to the solution set until a solution is reached.

3) At each If the solution set is feasible, the current item is kept.

4) Else, the item is rejected and never considered again.

- Huffman Encoding

- Huffman coding is technique of compressing data to reduce its size without losing any of the details, it was first developed by David Huffman.
- Huffman coding is generally useful to compress the data in which there are frequently occuring characters.
- Huffman coding is a famous Greedy Algorithm.
- It is used for the dta cossless compression of data.



Initial String.

- Prefix Rule :

- Huffman coding implementation a rule known as a prefix rule.
- This is to prevent the ambiguities while decoding
- It ensures that the code assigned to any characters is not prefix of the code.

- major steps in Huffman coding :

- There are two major steps in Huffman coding.

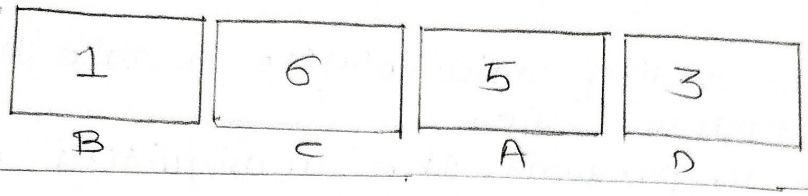
1) Building a Huffman Tree from the input characters.

2) Assigning code to the characters by traversing the Huffman tree.

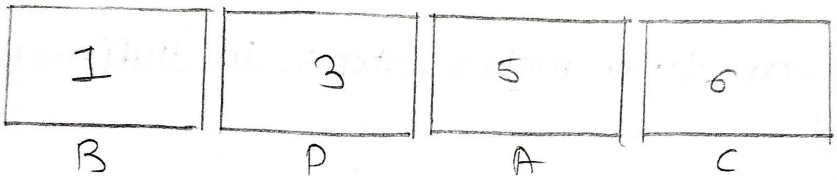
- How does the Huffman coding works :

- Suppose the string on the side page is to be sent over network.

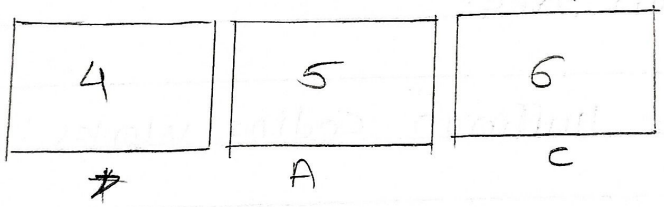
- each character occupies 8 bits there are a total of 15 characters in the above string Thus, a total of 8 * 15 = 120 bits are required to send this string
- using the Huffman coding first creates a tree using the frequencies of the characters and then generates code for each

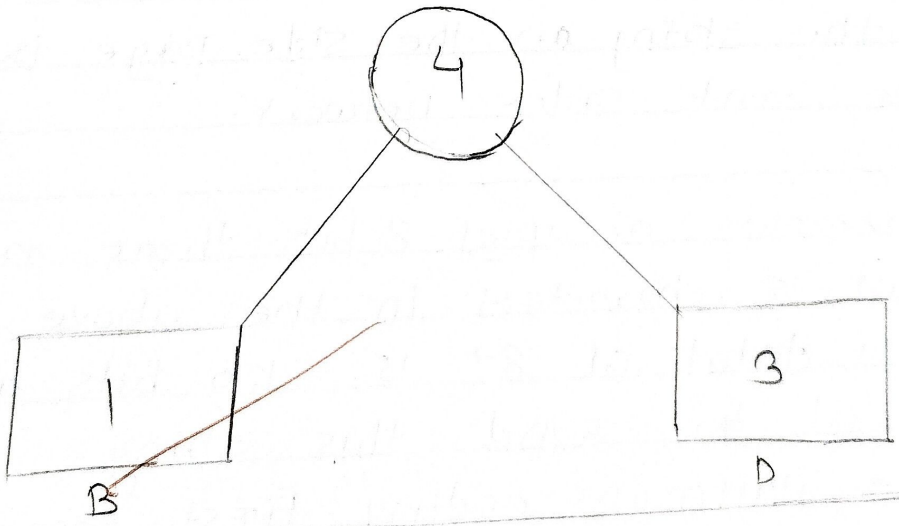


frequency string



Characters sorted according to frequency





Getting the sum of the least To

character.

- Once the data is encoded, it has to be decoded. Decoding is done using the same tree.
- Huffman coding is done with the help of the following steps:

Step1 : Calculate the frequency of each character in the string.

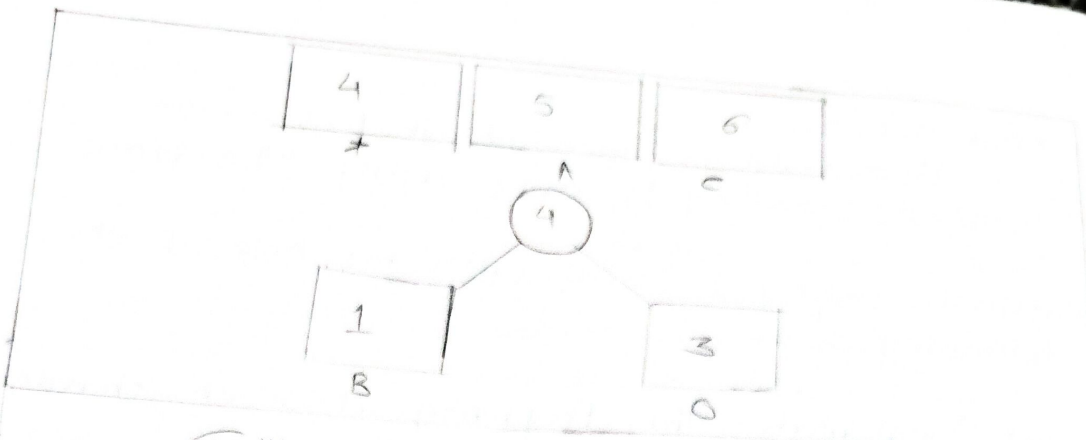
Step2 : Sort the characters in increasing order of the frequency these are in a priority queue Q.

Step3 : make each unique character as a leaf node.

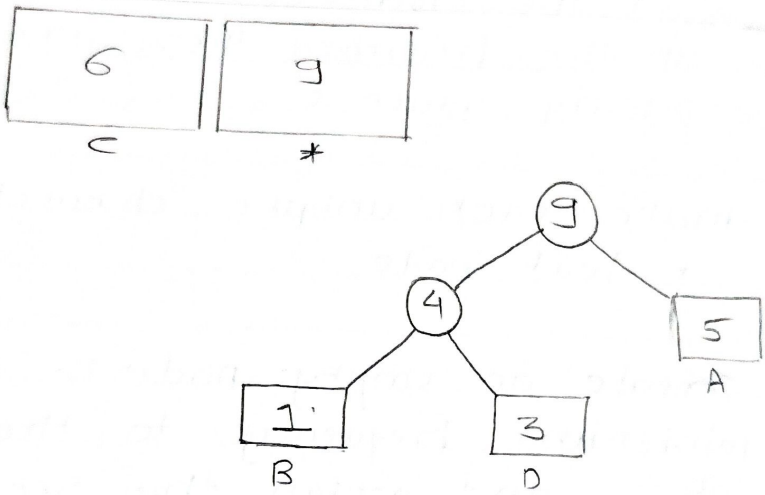
Step 4 : Create an empty node z. Assign the minimum frequency to the left child of z. and assign the second minimum frequency to the right child of z. set the value of the z as the sum of the above two minimum frequencies.

Step5 : Removes these two minimum frequence from Q and add the sum into the list of frequencies (* denote the maternal nodes in the figure above.)

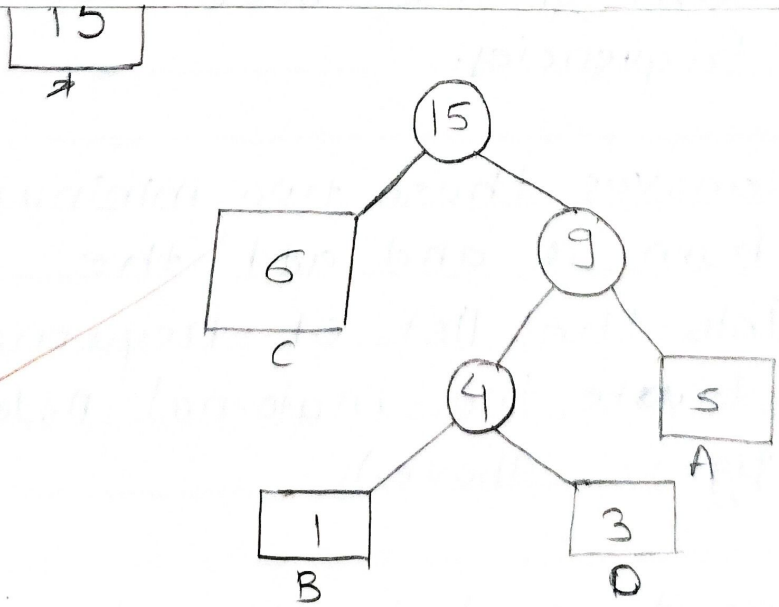
Step6 : Insert z into the tree.



Getting the sum of least no



Repeat step's 3-5 for all characters.



Repeat step 3 to 5 for all characters

Step 7: Repeat step 3 to 5 for all the characters.

For sending the string which on next page over a network, we have to send the tree as well as above compressed code. the total size is given by table below of string on next page.

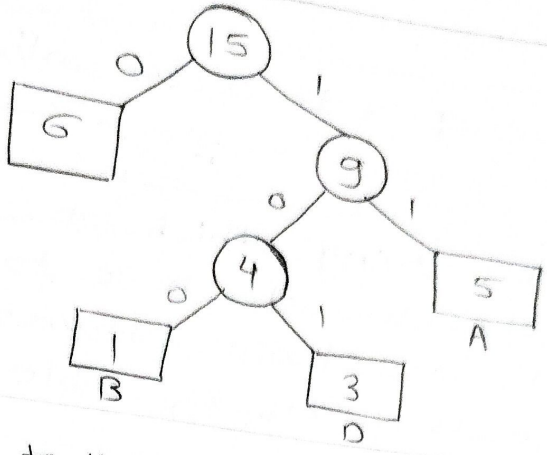
Without encoding, the total size of the string was 120 bits after encoding the size is reduced to 32 + 15 + 28 = 75.

- Example

– A file contains the following characters with the frequencies as shown. If huffman coding is used for data compression, determine.

1.) Huffman code for each ~~each~~ character
2) Average code length.
3) length of Huffman encoded msg.

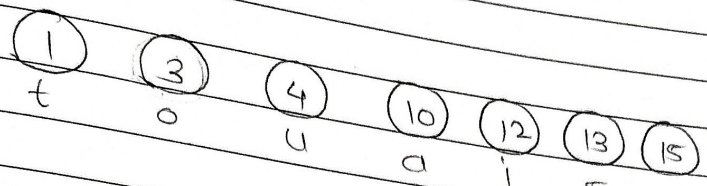
characters	frequencies
a	10
e	15
i	12
o	3
u	4
s	13
t	1



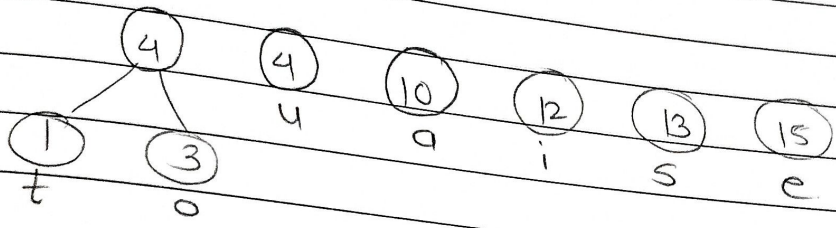
Assign 0 to the left edge & 1 right edge

Characters	Frequency	code	size
A	5	11	5*2=10
B	1	100	1*3=3
C	6	0	6*1=6
D	3	101	3*3=9
4*B=32 bits	15 bits		28 bits

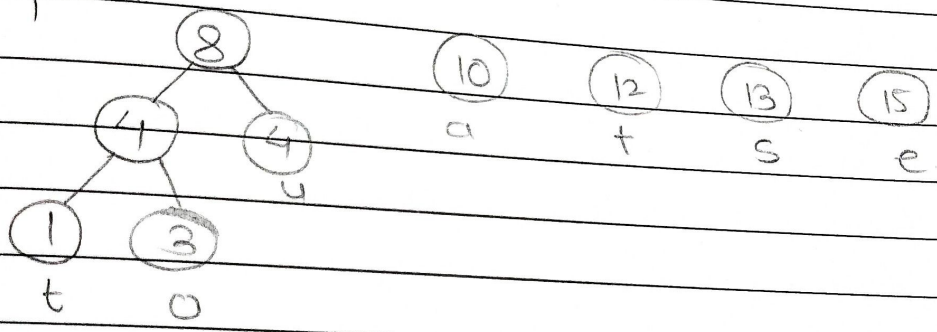
Step 1:



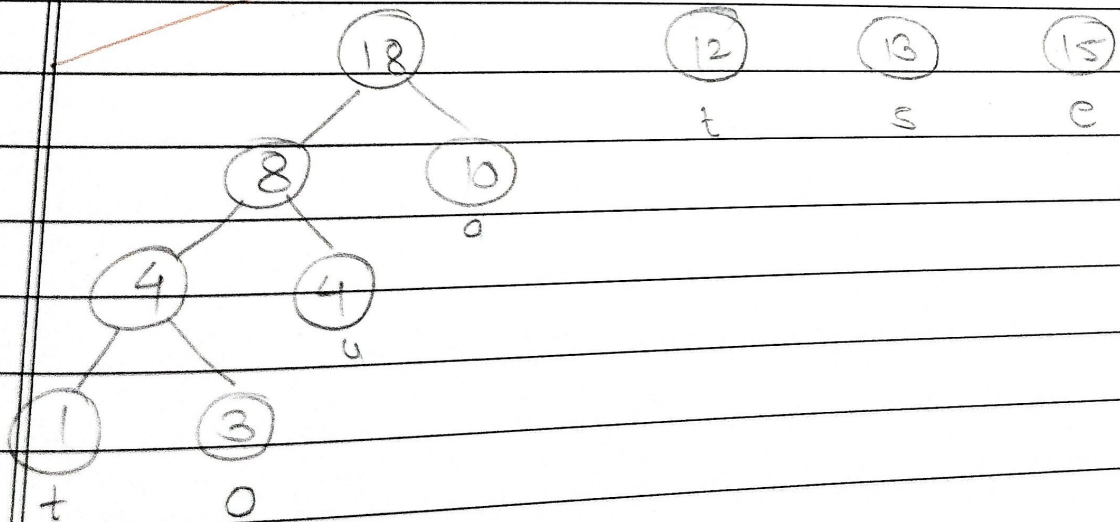
Step 2:

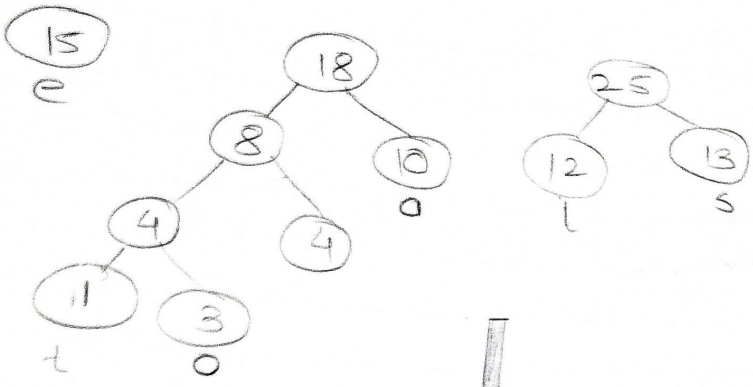


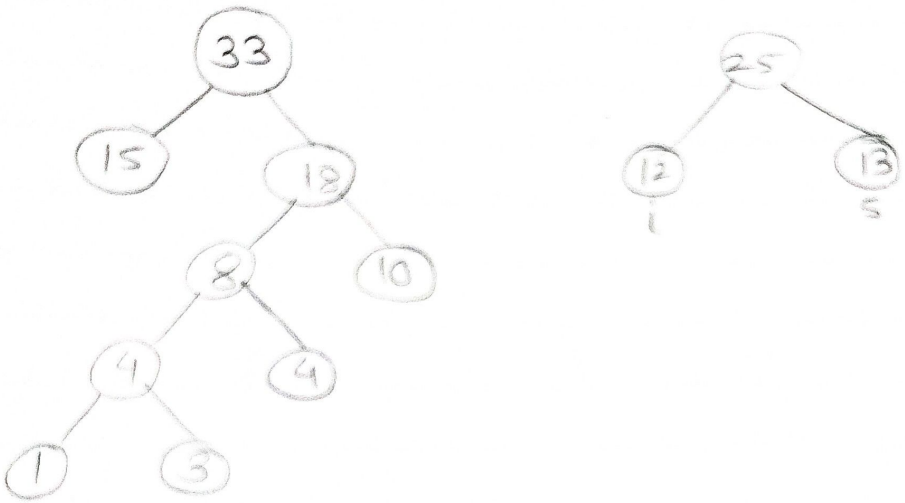
Step 3:



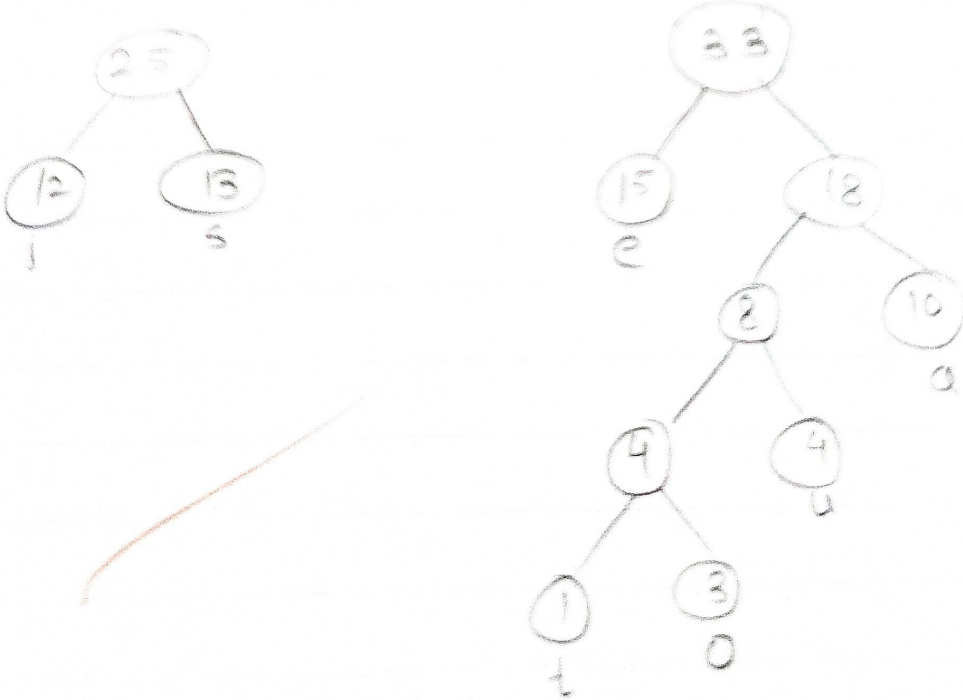
Step 4:

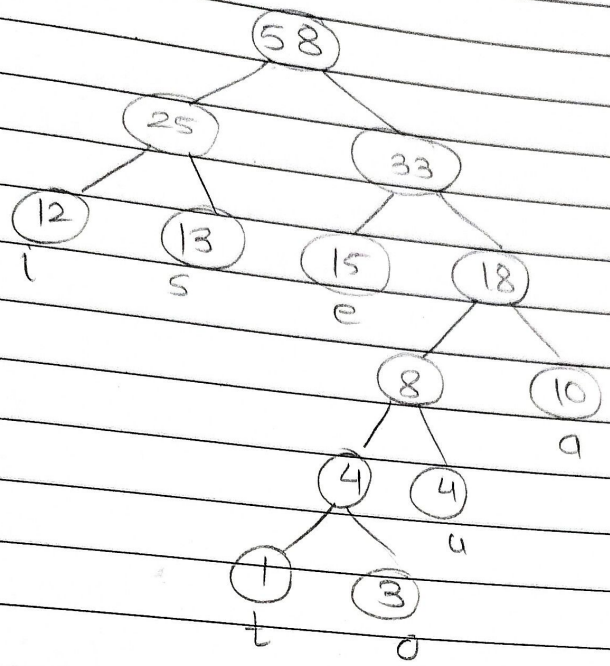



15
e
18
8
10
a
4
4
11
t
3
o
25
12
l
13
s

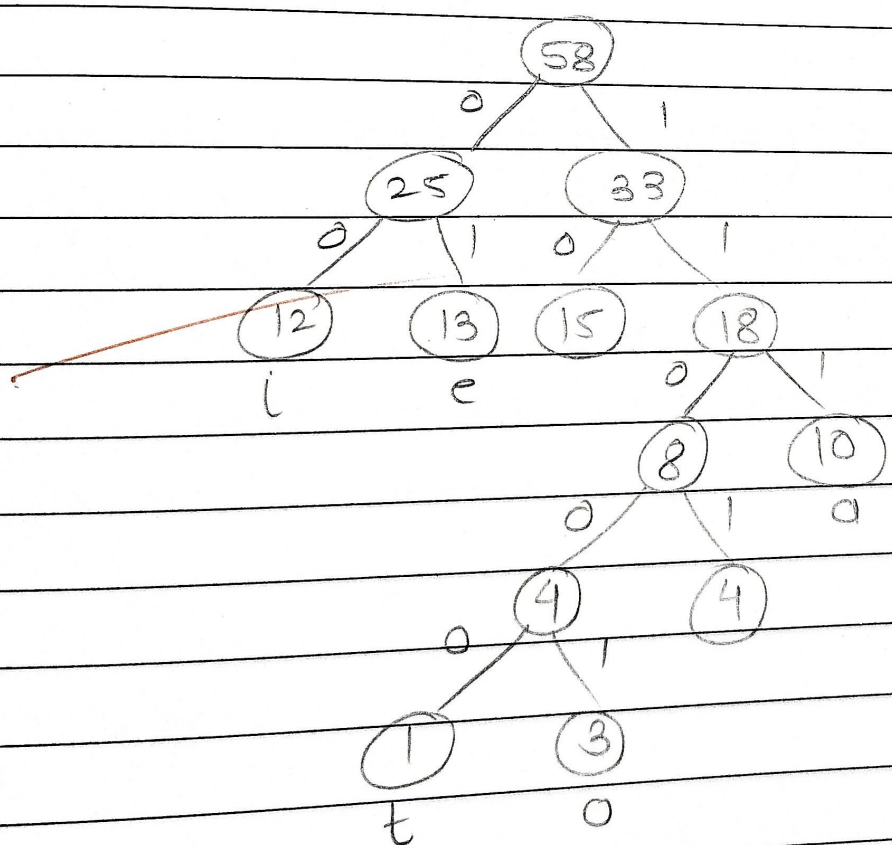

33
15
18
8
10
4
4
1
3
25
12
l
13
s

Step 7 :


25
12
l
13
s
33
15
e
18
8
10
a
4
4
1
t
3
o



After assigning weight to all the edges, the modified Huffman Tree is:-



Huffman Tree :

To write Huffman Tree code for any character, traverse the Huffman tree from root node to the leaf node of that character. Following this rule, the Huffman code for each character is :

a = 111
e = 10
i = 00
o = 110011
u = 1101
s = 01
t = 11000

Time Complexity :

- The time complexity analysis of Huffman coding as follows-
  - extract () is called 2 * (n-1) times if there are n nodes.
  - As extract () calls minHeapify () it takes O (log n) time.

- Conclusion :

In this way we have explored concept of Huffman Encoding using greedy method.

- Program : Huffman Encoding (Greedy strategy)

```
import heapq

class node:
    def __init__(self, char, freq):
        self.char = char
        self.freq = freq
        self.left = None
        self.right = None.

    def __lt__(self, other)
        return self.freq < other.freq

def build_huffman_tree(text):

    freq = {}
    for ch in text:
        freq[ch] = freq.get(ch, 0) + 1

heap = [Node(ch, fr) for ch, fr in freq.items()]
heapq.heapify(heap)

While len(heap) > 1:
    left = heapq.heappop(heap)
    right = heapq.heappop(heap).

merged = Node(None, left.freq + right.freq)
merged.left = left
merged.right = right.
```

```
        heapq.heappush(heap, merged)
    return heap[0]

def generate_codes(node, code="", codes={}):
    if node is None:
        return
    if node.char is not None:
        codes[node.char] = code
    generate_codes(node.left, code + "0", codes)
    generate_codes(node.right, code + "1", codes)
    return codes

def huffman_encoding(text):
    root = build_huffman_tree(text)
    codes = generate_codes(root)
    encoded_text = "".join(codes[ch] for ch
                           in text)
    return codes, encoded_text

text = "greedy shategy"
codes, encoded = huffman_encoding(text)
```

Output :

Characters huffman codes :

g : 110
r : 10
e : 00
d : 110
y : 1111
s : 0100
t : 0101
a : 001

Encoded string :

1101000001110111101101000101100
0101011101111.

```
print("character Huffman codes:")
for ch in codes:
    print(f"{ch} : {codes[ch]}")
print("\nEncoding string:", encoded)
```

Conclusion :

In this way we have explored concept of Huffman encoding using greedy method.

Practical No: 03

- Aim: Write a program to solve a fractional knapsack problem using a greedy method.
- Prerequisite:
  1. Basic of python programming.
  2. Concept of Greedy method.
  3. Fractional knapsack problem.
- Theory:

1) Greedy Method:

- A greedy algorithm is an approach for solving a problem by selecting the best option available at the moment. It does not worry whether the current best result will bring the overall optimal result.
- The algorithm never reverses the earlier decision even if the choice is wrong. It works in a top-down approach
- This algorithm may not produce the best result for all the problems. Its because it always goes for the local best choice to produce the global best result.

- Advantages of Greedy Approach:

The algorithm is easier to describe.

- This algorithm can perform better than other algorithms (but not in all cases).

• Drawback of Greedy approach:

- Does not produce the optimal solution.
- for ex. suppose we want to find the longest path in the graph below from root to leaf

• Greedy Algorithm:

- you are

1) To begin with, the solution set (connecting answers) is empty.
2) At each ~~step~~ an item is added to the solution set until a soln is reached.
3) If the solution set is feasible, the current item is kept.
4) Else, the item is rejected and never considered again.

• knapsack problem:

You are given the following:

- A knapsack with limited weight capacity.
- Few items each having some weight & value.

• The problem states-

Which items should be place into the knapsack such that:

- The value or profit obtained by putting the items into the knapsack is maximum.
- And the weight limit of the knapsack does not exceed possible.

- Fractional knapsack problem using Greedy method.
- Fractional knapsack problem is solved using greedy method in the following step:-

Step 01:-

For each item, compute its value/ weight ratio.

Step 02:

Arrange all the items in decreasing order of their value/weight ratio

Step 03:

Start putting the items into the knapsack begining from the item with the highest ratio. put as many items as you can into the knapsack.

Solution

step 01 :

Compute the value/weight ratio for each item.

Items	Weight	value	Ration
1	5	30	6
2	10	40	4
3	15	45	3
4	22	77	3.5
5	55	90	3.6

step 02 : Sort all the items in decreasing order of their value/weight ration -

I1	I2	I5	I4	I3
(6)	(4)	(3.6)	(3.5)	(3)

step3 : Start filling the knapsack by putting the items into it one by one.

knapsack Weight	Items in knapsack	cost
60	$\emptyset$	0
55	I1	30
45	I1, I2	70
20	I1, I2, I6	160

Now,

- knapsack weight left to be filled is 20kg but item-4 has a weight of 22kg.
- since in factorial knapsack problem, even the fraction of any item can be taken.
- so, knapsack will contain the following items -

$\langle I_1, I_2, I_5 (20/22) I_4 \rangle$

Total cost of the knapsack.

$= 160 + (20/27) \times 77$

$= 160 + 70$

$= 230$ units.

- Time complexity:

- The main time taking step is the sorting all items in decreasing order of their value/weight ratio.

- If the items are already arranged in the required order, then while loop takes $O(n)$ time.

- Therefore, total time taken including the sort is $O(n \log n)$.

- The average time complexity of Quick Sort is $O(n \log n)$.

• Fractional knapsack using Greedy method.

```
class Items
    def __init__(self, value, weight):
        self.value = value
        self.weight = weight
        self.ratio = value/weight
def fractional_knapsack(values, weights,
                        capacity):
    item = []
    for i in range(len(values)):
        items.append(item(values[i], weight[i]))
    items.sort(key = lambda x : x.ratio,
    reverse = True)

    total value = 0.0
    knapsack_items = []

    for items in items:
        if capacity == 0:
            break
        if item weight <= capacity:

total value += item value.
capacity -= item.weight
knapsack.items append ((items.value, item.
                        weight, 1))


else
fraction = capacity / item weight
total.value += item value * fraction
```

Output:

for values [60, 100, 120], weights [10, 20, 30]
capacity 50:

maximum value in knapsack = 240.0

Items taken (value, weight, fraction):

value : 60, weight : 10, taken : 100%.

value : 100, weight : 20, taken : 100%.

value : 120, weight : 30, taken : 66.67%.

```
        knapsack_items.append((item.value, item.weight, fraction))
        capacity = 0
    return total_value, knapsack_items

# Example
values = [60, 100, 120]
weights = [10, 20, 30]
capacity = 50

max_value, items_taken = fractional_knapsack(values, weights, capacity)

print("maximum value in knapsack =", max_value)

print("\n Items taken (value, weight, fraction):")

for v, w, f in items_taken:
    print(f" value: {v}, weight: {w}, taken: {f*100:.2f}%")
```

Conclusion:

In this way we have explored concept of fractional knapsack using greeding method.

Practical No: 04

- Aim: Write a program to solve a 0-1 knapsack problem using dynamic programming or branch & bound strategy.

- Prerequisite:

1) Basic of python programming.
2) Concept of dynamic programming.
3) 0/1 knapsack problem.

- Theory:

- Dynamic Programming:

- Dynamic programming is also used in optimization problems. like divide & conquer method. dynamic programming solves problems by combining the solutions of subproblems.

- Dynamic programming algorithm solves each sub-problem just once & then saves its answer in a table, thereby avoiding the work of recomputing the answer every time.

- Two main properties of problem suggest that the given problem can be solved using Dynamic programming. these properties are overlaping, subproblems and optimal.

substructure.

for ex. binary search does not have overlapping such problem whereas recursive program of fibonacci numbers have many overlapping sub-problems.

- steps of dynamic programming Approach.

- Dynamic programming algorithm is designed using the following four steps:
  - characterize the structure of an optimal solution.
  - Recursively define the value of an optimal solution.
  - compute the value of an optimal solution. typical in a bottom up fashion.
  - construct an optimal solution from the computed information.

- Applications of Dynamic programming.
  - matrix chain multiplication.
  - longest common subsequence.
  - Travelling salesman problem.

- knapsack problem.

:- You are given the following:

- A knapsack with limited weight. capacity.
- few items each having some weight & value.

- The problem states:

Which items should be placed into the knapsack such that -

- The value or profit obtained by putting the items into the knapsack is maximum.
- And the weight limit of the knapsack does not exceed.

- knapsack problem variants:

knapsack problems has the following two variant

1.) Fractional knapsack problem.

2) 0/1 knapsack problem.

- 0/1 knapsack problem:

In 0/1 knapsack problem,

- As the name suggests, items are invisible here.

- We cannot take a fraction of any item.
- We have to either take an item completely or leave it completely.
- It is solved using a dynamic programming approach.
- 0/1 knapsack problem using Greedy method:

Consider -
- knapsack weight capacity = W
- No of items each having some weight and value = n.

- 0/1 knapsack problem is solved using dynamic programming in following steps.

step 01:
- Draw a table say. I with (n+1) numbers of rows and (W+1) no. of columns.

- fill all the boxes of 0th row & 0th column with zeroes as shown.

	0	1	2	3	---	W
0	0	0	0	0	---	0
1	0					
2	0					
:	---					
:	0					
n	0					

Table

step 02:

start filling the table row wise top to bottom from.

left to right use the following formula-

$$T(i,j) = \max \{ T(i-1,j), value_i + T(i-1, j - weight) \}$$

Here, $T(i,j)$ = maximum value of the selected items if we can take items 1 to i and have weight restrictions of j.

- step 03:

- To identify the items that must be put into the knapsack to obtain the maximum profit,
- consider the last column of the table.
- start scanning the entries from bottom to top
- After all entries are scanned, the marked labelled represents the items that must be put into the knapsack.

- Time complexity:

- each entry of the table required constant

time $\phi(1)$ for its computation.

- it takes $\phi(nw)$ time to fill $(n+1)$ $(W+1)$ table entries.
- It takes $\phi(n)$ time for tracing the solution since tracing process traces the n rows.
- Thus, overall $\phi(nw)$ times is taken to solve 0/1 knapsack problem using dynamic programming.

- 0-1 knapsack problem using dynamic programming.

```
def knapsack(values, weights, capacity):
    n = len(values)

    dp = [[0 for _ in range(capacity+1)]
          for _ in range(n+1)]

    for i in range(1, n+1):
    for w in range(capacity+1):

        if weights[i-1] <= w:

dp[i][w] = max(values[i-1] + dp[i-1]
[w - weights], dp[i-1][w])

else:
```

```
            dp[i][w] = dp[i-1][w]
    res = dp[n][capacity]
    w = capacity
    items_taken = []
    for i in range(n, 0, -1):
        if res <= 0:
            break
        if res == dp[i-1][w]:
            continue

        else:
        items_taken.append(i-1)
        res = res - values[i-1]
return dp[n][capacity], items_taken.

values = [60, 100, 120]
weights = [10, 20, 30]
capacity = 50

max_value, items = knapsack(values, weights,
    capacity)

print("maximum value =", max_value)
print("Items taken: (0-idx):", items)
```

- Conclusion:

In this way we have explored 0/1 knapsack using dynamic approach.

Practical No: 05

Aim: Design n-queens matrix having first queen placed. Use backtracking to place remaining queens to generate the final n-queens matrix.

Objectives:

1) To understand the concept of backtracking.
2) To learn how to represent and solve constraint satisfaction problem.
3) To implement an efficient recursive soln to the N-queens problem.
4) To analyze the time complexity of a backtracking algorithm.

Theory:

The N-Queens problem is a classic combinational problem in which the goal is to place N queens on an NXN chessboard such that no two queens attack each other.

That means:

- No two queens share the same row.
- No two queens share the same column.

- No two queens share the same diagonal

- Backtracking concept:

- Backtracking is a systematic method of trying out different sequences of decisions, until you find one that "works".

- It is used for problems involving constraints and combinatorial searches.

- In this problem.

1) We can place a queen in a valid position in the current column.

2) We recursively attempt to place the next queen in the next column.

3) If at any point we find no valid position. We backtrack. - i.e. remove the previous queen and try the next possibility

* Algorithm:

Step 1: start

Step 2: Define a function solveNQueen (board, col, n) to place queens column-wise.

step 3 : If col >= n print the board all queens are placed successfully.

step 4 : for each row i in column col :

- check if placing a queen at (i, col) is safe.
- If safe, place the queen board [i] [col] = 1
- Recursively call solveNqueen (board, col+1, n).
- If placing the queen doesn't lead to a soln, remove it (board [i] [col] = 0) - Backtrack.

step 5 : continue until all possible placements are tried.

step 6 : Stop

* checking function (issafe)

This function ensures that placing a queens at position (row, col) does not lead to attacks from previously placed queens.

Check three conditions :

1) left Row : No queen on the left side in the same row.

2> upper left Diagonal : No queen on the upper left diagonal.

3> lowler left diagonal : No queen on the lowler left diagonal.

- Program :

```
class NQBacktracking:

    def __init__(self, x, y):
        """ self.ld is an array where its indices
        indicate row - col + N - 1
        (N-1) is for shifting the difference to
        store Negative indices"""

        self.ld = [0] * 30

        """
        self.rd is an array where its indices indicate
        row + col and used to check whether
        a queen can be placed on right diagonal
        or not"""
        self.cl = [0] * 30

        """ Initial position of 1st queen"""
        self.x = x
        self.y = y

    def printSolution(self, board):
        """ A utility function to print solution"""
        print("""N queen Backtracking solution:\n
```

```
Given Initial position of 1st queen at
row:",
self.x,
"column:",
self.y,
"\n"
)
for line in board:
    print (" ".join(map(str, line)))

def solveNQUtil(self.board, col):
""" A recursive utility function to solve
N Queen problem """

if col = self.y:
    return self.solveNQUtil(board, col+1)

for i in range(N):
    if i = self.x
        continue.
if (self.ld[i - col + N - 1] ≠ and self.rd
[i + col] ≠ 1) and self.cl[] ≠ 1:

board[i][col] = 1
self.ld[i - col + N - 1] = self.rd[i + col] = self.
d[i] = 1

if self.solveNQUtil(board, col + 1)
    return True
```

```
        board[i][col] = 0
        self.id[i-col+N-1] = self.id[i+col]
        = self.cl[i] = 0
        return false

    def solveNQ(self):
        board = [[0 for-in range(N)] for-in rang(N)]
        board[self.x][self.y] = 1

    if not self.solveNQUtil(board, 0):
    print("solution does not exist")
        return false
        self.printsolution(board)
        return True

    if ... name ... = "...main..":
        N = 8
        x, y = 3, 2
        NQBT = NQBacktracking(x, y)
        NQBT.solveNQ()
```

"""

OUTPUT:

N Queen: Backtracking solution:

Given initial position of 1st queen at row: 3 column: 2

```
1 0 0 0 0 0 0
0 0 0 0 0 1 0
0 0 0 0 0 0 1
0 0 1 0 0 0 0
0 0 0 0 0 0 0
0 0 0 1 0 0 0
0 1 0 0 0 0 0
```

conclusion :

In this way successfully implemented Naueens problem backtracking.